A Comprehensive Analysis and Optimization of 2D Human Pose Estimation Using Heatmap Regression

Practical Work in AI (344.031)

Philipp Gatterbauer
K12102712

Johannes Kepler University Linz

Abstract

This report presents the development and evaluation of a 2D human pose estimation model using heatmap regression. The study explores the impact of preprocessing, loss functions, and postprocessing techniques, identifying effective approaches for improving pose estimation accuracy.

1 Introduction

1.1 Motivation

Human Pose Estimation (HPE) is the task of detecting the positions of keypoints corresponding to anatomical landmarks on a human body from images or video frames. These keypoints are then used to infer the overall posture of the person in the image [1]. HPE has a wide range of applications such as action recognition, video surveillance, AR/VR, healthcare, and gaming [2]. Advancements in deep learning have significantly improved performance of HPE-models. However, there are still challenges which will require an extensive amount of research such as occlusions or unusual, complex poses of humans [3].

In this practical work, we will develop a model suitable for 2D single person pose estimation using heatmap regression. Throughout this process, we will analyze and compare various components and methodologies involved. This comparison will help identifying effective approaches and point out those that may be less successful.

1.2 Research Questions

1. How does preprocessing impact the results for HPE?
How does the selection method for bounding boxes around individuals influence the final outcomes?

2. What are suitable loss functions for heatmap regression in HPE?
Which loss functions are effective for this task and why?

3. Which postprocessing techniques are appropriate for HPE?
What types of postprocessing techniques that can be applied and how do they affect the results?

1.3 Outline

In Section 2, the material, including the dataset's images and annotations and the methods used, are described. The section about methods includes the theoretical background and the architecture of the network used. Section 3 covers the implementation, including the preparation of annotations and images for training, as well as details about training, prediction and the implementation of postprocessing methods. Section 4 provides a guide on using the pose estimation model implementation developed in this project. The experiments conducted to address the research questions are presented in Section 5. Finally, Section 6 offers conclusions and potential future improvements.

2 Material and Methods

2.1 Material

MPII Human Pose Dataset [4] The MPII Human Pose Dataset is specifically designed for training and evaluating 2D human pose estimation methods. It contains around 25,000 images and more than 40,000 annotated individuals. Of these, approximately 18,000 images are allocated for training and 7,000 for testing. The images are collected from everyday activities, which makes the dataset highly diverse in terms of poses, clothing, and occlusions of body parts. Each training image contains one or more annotated individuals. Each annotation includes the positions of 16 body joints, along with information about whether the body part is occluded. Additionally, the rough position of each person, together with their scale relative to 200 pixels, is provided [4]. Due to its thorough annotations and complexity, this dataset serves as a benchmark for evaluating the performance of pose estimation algorithms.



Figure 1: Examples from the MPII dataset including the body joint coordinate annotations [4]

2.2 Methods

2.2.1 Theoretical Background

You Only Live Once (YOLO) [5]
YOLO, which is short for "You Only Look Once", is an object detection system which handles object detection as a single regression problem. Traditional approaches before YOLO repurposed classifiers to perform object detection using complex pipelines. YOLO simultaneously predicts bounding boxes and the probability to which one of 20 categories each box belongs to in a single evaluation. This CNN architecture allows YOLO to achieve real time speeds when predicting bounding boxes with high accuracy [5].
In Section 3.2 we describe how YOLO can be used to extract bounding boxes of humans from images for the task at hand.

Convolutional Neural Network
A Convolutional Neural Network (CNN) is a type of deep learning model commonly used in areas like image and signal processing. CNNs take advantage of the spatial structure of signals, such as images, by using specialized layers like convolutional layers, pooling layers, and fully connected layers. In convolutional layers, learnable filters are applied to the input, allowing the detection of important features, known as activation maps, such as edges or shapes. Pooling layers reduce the spatial size of these activation maps while preserving the most important information. Fully connected layers combine the features extracted by previous layers to produce the final output, typically used for classification or regression tasks [6].

Residual Network (ResNet) [7]
ResNet is a neural network architecture designed to solve the degradation problem in deep networks, where adding more layers increases training error. It uses residual connections, allowing gradients to bypass certain layers and flow between earlier and later ones. This technique prevents the vanishing gradient problem and reduces the degradation issue, making deep networks easier to train and optimize [7].
There are different pre-trained ResNet models with varying numbers of layers. In this project, the ResNet-50 model, which contains 50 layers, is used as a backbone of the network.

Supervised Learning
The goal in Supervised Machine Learning is learning the relationship between inputs and outputs (labels) based on a set of paired input-output training samples [8].
In this project, the model is trained to map input images to annotated body joint coordinates, converted to heatmaps, learning to predict joint positions from extracted image features.

Percentage of Correct Keypoints (PCK)
The PCK metric is frequently used to evaluate HPE models. Detected keypoints are counted as correct if they are within a normalized threshold distance of the ground truth keypoints [3]. For the MPII dataset, the threshold is based on the height of a box around the head (PCKh@0.5). A predicted keypoint is considered correct if it lies within a distance defined by:

$$\text{Threshold} = 0.5 \times \textit{head_length}$$

of the actual coordinate's location [4].

2.2.2 Network Architecture

The model architecture is based on the Deconvolution Head Network presented by Xiao *et al.* (2018), which was originally designed as a baseline for Human Pose Estimation models [9]. This architecture is adopted for this project because the primary focus is not on the model architecture itself but on exploring and evaluating different factors that may influence the effectiveness of pose estimation with heatmap regression.
The architecture begins with an input layer that accepts images with dimensions of 224x224x3. This input is then processed through the backbone, which utilizes the ResNet-50 model, pretrained on the ImageNet dataset. As more layers mean longer training and inference time, ResNet-50 is choosen as a balance between accuracy and computational efficiency.

Following feature extraction, the network enters a bottleneck and upsampling phase. Here, deconvolution layers (Conv2D transpose) progressively increase the spatial resolution from 7x7, to 14x14, 28x28, and finally 56x56. Each upsampling layer is accompanied by batch normalization and ReLU activation to ensure stable training and introduce non-linearity. Finally, the network concludes with an output layer that consists of a Conv2D layer which outputs a 56x56x16 tensor, corresponding to 16 heatmaps, each of size 56x56.
This architecture balances feature extraction capabilities and computational efficiency and therefore short training times, making it suitable for keypoint localization tasks while allowing for iterative optimization during experimentation with other components of the process.

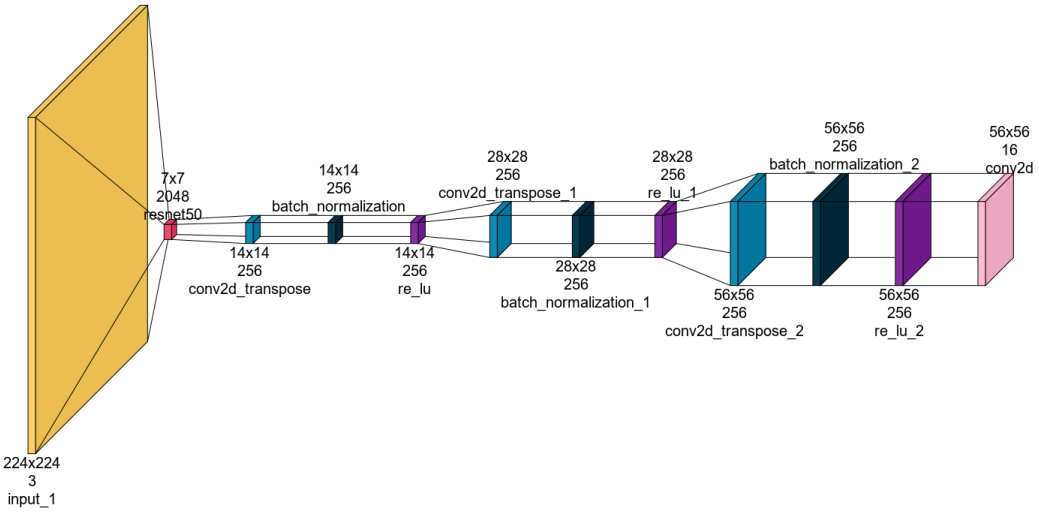


Figure 2: The network architecture. Plot created with visualkeras [10].

3 Implementation

3.1 Preparing Annotations

Each image in the MPII dataset is labeled as either belonging to the training set or the test set. The test set does not include annotations for keypoint coordinates. Additionally, there is no officially designated validation set provided with the dataset. Therefore, for all experiments, we use a validation set consisting of 2,958 images, following the split proposed by Thompson *et al.* [11]. This split is commonly adopted in literature [12, 13, 14]. The images are also labeled according to their suitability for single-person pose estimation. After filtering for single-person estimation, the dataset comprises 13,023 training images with 15,855 annotated persons, 1,936 images with 1,991 annotated individuals for validation, and 5,709 images with 7,247 annotated persons for testing. The annotations are extracted and concatenated to form a final shape of $(nr_images \times 32)$.

3.2 Preparing Images and Heatmaps

Approaches for pose estimation can be grouped into two parts:

- **Bottom-Up Approaches:** These start by detecting all body joints in the image and then assembling them into individual poses. These approaches are generally more efficient compared to top-down approaches [2].
- **Top-Down Approaches:** These begin by detecting each person in the image and then estimating the joint locations for each one. This method tends to be more accurate [2].

For this project, a top-down approach is implemented, starting with the detection of individuals in images.
When processing the images, we therefore first identify bounding boxes around individuals. In this project, two approaches are tested and compared in Section 5.1. One of these approaches is also commonly used in the literature and it involves applying an object detection algorithm like R-CNN-based methods [15].
In this project, the YOLO (You Only Look Once) object detection model is used. Despite being generally less accurate than region-based approaches, it is significantly faster [16].
We extract all found boxes classified as *person*. According to the MPII dataset's single person pose estimation guidelines, it is allowed to use approximate locations of individuals provided in the annotations [11].
We analyze all detected person bounding boxes, selecting the ones that encompass this point. If multiple boxes include the location, the one with the highest YOLO confidence score is chosen. For single person pose estimation, the data is filtered to ensure that individuals are well-separated, usually resulting in a single bounding box around the point. In cases where no bounding box is found after this selection process, an approximate bounding box is generated as outlined in Section 5.1.1. The selection process is described in detail in Listing 1.

```python
def detect_people_batch(model, paths, positions, scales):
    assert len(paths) == len(positions) == len(scales), "Mismatched input lengths."
    all_person_boxes = []
    all_filtered = []
    for i, (path, pos, scale) in enumerate(zip(paths, positions, scales)):
        print(f"Processing {i}: {path}")
        results = model(path, save=False)
        filtered = filter_people(results, pos, all_person_boxes)
        if not filtered:
            print(f"No person found at {pos}")
            approx_box = approximate_bounding_box(pos, scale)
            filtered = [{"image_path": path,
                "class_name": "person",
                "box_coordinates": approx_box,
                "confidence_score": 0}]
            print(filtered)
        all_filtered.append(filtered)
    return all_filtered, all_person_boxes

def filter_people(detections, position, all_person_boxes):
    """
    Filters detections to return only those for people near a person's position.
    """
    ...
```

Listing 1: Algorithm to find the best bounding box

Afterwards, the selected bounding box is made slightly bigger to provide more spatial context to the keypoint estimation model. The image is then cropped to this bounding box and padded to dimensions 224×224.

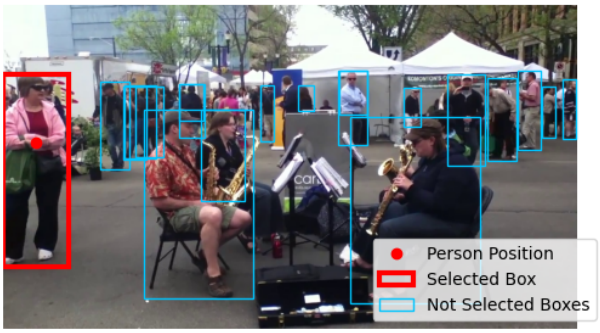


Figure 3: Selection of the Bounding Box.



Figure 4: Processed Image.

In the context of pose estimation, heatmap regression is generally preferred over direct coordinate regression. This way, regression on the probability values of each pixel is performed, which allows the model to learn the spatial context of the keypoints better compared to coordinate regression [2].

Using the padding and cropping information from the step before, the annotated coordinates are transformed from their original version, such that they fit onto the processed image. For each scaled coordinate, a heatmap is generated according to Equation 1 and Listing 2, using $\sigma = 2$.

$$\text{heatmap}[i, j] = \exp\left(-\frac{(i - x_{\text{scaled}})^2 + (j - y_{\text{scaled}})^2}{2 \cdot \sigma^2}\right) \tag{1}$$

```python
for idx, joint in enumerate(joints):
    if visibility[idx]:  # Check if the joint is visible
        x, y = joint
        # Scale x and y to match the heatmap size
        x_scaled = int((x / 224) * width)
        y_scaled = int((y / 224) * height)

        # Compute the heatmap
        heatmaps[idx] = np.exp(-((xx_grid - x_scaled) ** 2 + (yy_grid - y_scaled) ** 2) / (2 * sigma ** 2))
    else:
        # Set to zero if not visible
        heatmaps[idx] = np.zeros((height, width), dtype=np.float32)
```

Listing 2: Creating the heatmaps for one person

A 2D Gaussian distribution is created, where the peak is at the keypoint's location. If the coordinate is not visible in the original image, the heatmap is set to zero everywhere.

3.3 Training Details

The model depicted in Figure 6 is trained using the following specifications:

- Batch size: 16
- Number of epochs: 50
- Early stopping with a tolerance of 5 epochs is utilized to prevent overfitting
- A learning rate scheduler is applied, defined by:

$$\begin{aligned} &\text{For } n < 10, \quad \text{lr}(n) = 0.001, \\ &\text{For } n \geq 10, \quad \text{lr}(n) = \text{lr}(n-1) \times e^{-0.1}. \end{aligned}$$

Details about the tested loss functions can be found in Section 5.2.

3.4 Prediction

After training, the model is able to predict the 16 heatmaps, corresponding to each body joint. The heatmaps are converted into coordinates as depicted in Listing 3

```python
def heatmaps_to_coords(hmaps, orig_size=56, tgt_size=224):
    scale = tgt_size / orig_size
    coords = []
    for hmap in hmaps:
        # Find coordinates of the max value in the heatmap
        y_max, x_max = np.unravel_index(np.argmax(hmap), hmap.shape)
        # Scale coordinates from heatmap size to image size
        x_scaled, y_scaled = int(x_max * scale), int(y_max * scale)
        coords.append((x_scaled, y_scaled))
    return coords

# images_batch: shape (batch_size, 224, 224, 3)
preds_batch = model.predict(images_batch)
# preds_batch: shape (batch_size, 56, 56, 16)
coords_batch = [heatmaps_to_coords(pred) for pred in preds_batch]
# coords_batch[i]: (16, 2) for 16 coordinate pairs
```

Listing 3: Prediction of coordinates for one image batch

The resulting coordinates fit onto the preprocessed 224×224 image. They are finally transformed to their location on the original image using stored information from the preprocessing steps.

3.5 Postprocessing

In Section 5.3, results of two postprocessing techniques are discussed. This includes two methods.

Gaussian Smoothing of Heatmaps

In two dimensions, the Gaussian function is defined as

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \quad [17] \tag{2}$$

Given the set of predicted heatmaps $H = \{H_1, H_2, \ldots, H_n\}$ The smoothed heatmaps $\tilde{H}_i(x, y)$ are computed by performing a convolution of the Gaussian function with each heatmap individually, using $\sigma = 1$

$$\tilde{H}_i(x, y) = (H_i * G_\sigma)(x, y) \quad \text{for } i = 1, 2, \ldots, N \tag{3}$$

Pixel Refinement

Following Xiao *et al.* (2018), the coordinate from the highest heatmap peak is adjusted towards the second highest peak [9]. Given a predicted heatmap H, the coordinates are extracted by adjusting the heatmaps peak (x_1, y_1) towards the second highest peak (x_2, y_2) within a 5×5 pixel window around (x_1, y_1). Given the horizontal distance between the two peaks $\Delta x = x_2 - x_1$ and the vertical distance $\Delta y = y_2 - y_1$, the offset is calculated as:

$$\text{offset}_x = \alpha \cdot \Delta x = \alpha \cdot (x_2 - x_1) \tag{4}$$

$$\text{offset}_y = \alpha \cdot \Delta y = \alpha \cdot (y_2 - y_1) \tag{5}$$

Where $\alpha = 0.25$. The extracted coordinates are then calculated as:

$$x_{\text{adjusted}} = x_1 + \text{offset}_x \tag{6}$$

$$y_{\text{adjusted}} = y_1 + \text{offset}_y \tag{7}$$

4 User Guide

Running the Program

Before using the program, the user should clone the corresponding GitHub repository. The pose estimation model itself is made available on Google Drive and should be put into the `model` folder. The program allows the usage of the model with the best results described in Section 5. The user can execute the program from the command line with various options:

- `--input <input_type>`: Specifies the type of input to process.
  - `image`: Process a single image.
  - `video`: Process a video file.
  - `webcam`: Process a live webcam feed.

- `--path <path_to_input>`: Specifies the path to the input image or video file. This option is required if the input type is `image` or `video`.
- `--output_path <path_to_output>`: Specifies the output path for the processed video containing the visualization. This option is required if the input type is `video`.
- `--visualization <visualization_type>`: Specifies the type of visualization to display.
  - `coordinates`: Displays only the joint coordinates.
  - `skeleton`: Displays the joint coordinates with lines connecting them to resemble the human body bone structure.
- `--threshold <threshold_value>`: A float value between 0 and 1 to set a confidence threshold for the heatmaps. If a joint's heatmap value falls below this threshold, its coordinate will not be plotted. This option allows the user to control the confidence level required for the model to display a joint's location.

Example Commands

Below are some example commands to run the program:

1. Predict on a single image with skeleton visualization and a threshold of 0.1:

```
python pose_estimation.py --input image --path path/to/image.jpg
--visualization skeleton --threshold 0.1
```

2. Predict on a video with coordinates visualization and a threshold of 0.2:

```
python pose_estimation.py --input video --path "path/to/video.mp4"
--output_path "path/to/output.mp4"
--visualization skeleton --threshold 0.2
```

3. Predict using the live webcam feed with skeleton visualization and a threshold of 0.1:

```
python pose_estimation.py --input webcam
--visualization skeleton --threshold 0.1
```

After making a prediction, the program will ask the user if they want to process more images or videos. Please note that the model is not optimized for real-time processing, so the speed of processing webcam input can vary depending on your hardware. With visualization enabled, the model currently runs at about 12 frames per second (fps) on an Nvidia RTX 3070 GPU.

5 Experimental Results

Experimental Setup

The experimental approach involves a sequential testing process. Initially, we compare two preprocessing methods to select bounding boxes. The models here use the Mean Squared Error as loss function and no preprocessing techniques. The superior method is then used to compare two different loss functions. The best-performing loss function is finally assessed with two postprocessing techniques. At each stage, the more effective technique advances to the next level, ensuring optimization of the process. The model

that performs the worst in the first step serves as a baseline for subsequent experiments, providing a reference point to measure the impact of each improvement.
The tables presented in this section show the accuracy of detection for the joints individually, evaluated on both the training and validation set. Also, the overall accuracy (Total) is given, which refers to the total percentage of keypoints that were accurately predicted. A point is counted as correctly predicted if it falls within a 0.5 threshold of the head segment length (PCKh@0.5) from the ground truth keypoint. The accompanying plots illustrate, how the PCKh value (Detection Rate) changes as this threshold (Normalized Distance) varies. The smaller the threshold, the closer the predicted keypoints must be to the ground truth keypoints in order to be considered correct. These curves are shown only for the validation set, as it is more informative because the model has never encountered these images during training. The AUC (Area under the Curve) measures the integral of the PCKh curve over this range of thresholds.
All plots shown in this section were created with the help of code from [18], which mainly uses the evaluation toolkit provided by the creators of the MPII dataset.

5.1 How does preprocessing impact the results for HPE?

5.1.1 Approximated Bounding Box

A rough bounding box is obtained using the annotated position coordinates and scale of the person relative to a height of 200 pixels. The bounding box is centered at the person's position. The corner coordinates for the bounding box are calculated by determining the top-left and bottom-right points based on half the width and half the height adjustments from the center position. These calculations ensure the bounding box scales with the person's size and remains within the dimensions of the image. The advantage for this method is, that the algorithm for finding the bounding box is independent of another model. However, there are notable drawbacks: the bounding box may lack accuracy, particularly if the person is positioned horizontally, due to its vertical orientation. Moreover, while suitable for the MPII dataset where each individual's position is predefined, this approach would not be viable in real-world applications without an accompanying human detection system.

5.1.2 YOLO Person Detection

The use of YOLO object detection to determine bounding boxes is described in 3.2. The advantage of using YOLO is that it is known for being fast at inference, while still achieving high accuracy, making it one of the top object detection methods available today [5].

5.1.3 Preprocessing Results

Table 1: PCKh@0.5 Scores: Preprocessing techniques

Method	Head	Shoulder	Elbow	Wrist	Hip	Knee	Ankle	UBody	Total	AUC
Approximated BBox (Train)	97.6	98.2	93.9	86.8	96.7	88.7	57.1	93.0	89.8	60.0
Approximated BBox (Val)	88.1	81.6	63.9	53.0	67.1	52.1	29.8	66.2	63.9	34.3
YOLO BBox (Train)	97.5	96.6	90.6	85.8	92.8	85.1	79.5	91.0	90.3	58.2
YOLO BBox (Val)	92.7	86.6	76.5	69.4	75.1	67.1	62.5	77.6	76.5	44.6

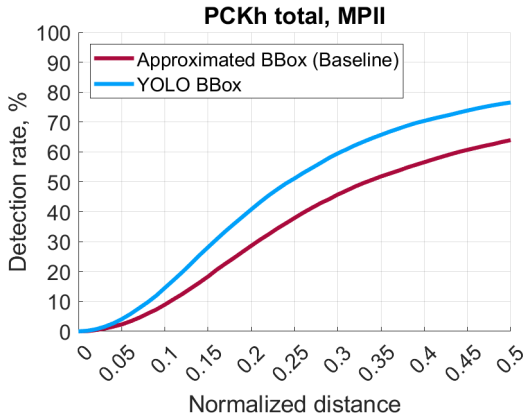


Figure 5: Preprocessing Results (Val): PCKh scores at varying thresholds

The results demonstrate that YOLO significantly outperforms the approximated bounding box method across all body parts on the validation set. This is particularly evident in the detection of limb joints, which are often truncated when using the approximated bounding box approach. This shows that the selection method for selecting bounding boxes is crucial for achieving good results. Notably, while the approximated bounding box performs worse on the validation set, it scores higher than YOLO on the training set. This is likely because the model has learned to adjust for errors in the training data, such as body parts being cut off in the images, but cannot generalize these corrections to the validation set, where cut off limbs remain unpredictable.

5.2 What are suitable loss functions for heatmap regression in HPE?

5.2.1 Mean Squared Error (MSE)

In pose estimation, the Mean Squared Error (MSE) is frequently employed to measure the average squared differences between predicted and actual values. One significant limitation of MSE is its sensitivity to outliers, as errors are squared, emphasizing larger discrepancies [19].
The MSE is defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2, \quad [19] \tag{8}$$

In our case, $\hat{y}_i$ represents the predicted heatmaps and y_i represents the ground truth heatmaps. The MSE is computed for each pixel in the heatmaps across all joints in the corresponding batch. For each joint, the MSE squares the difference between pixels of the

ground truth and corresponding predicted heatmap pixels, summing these squared differences, and then averaging them over the number of all pixels and joints which corresponds to $n = \text{batchsize} \times \text{heatmapsize} \times \text{heatmapsize} \times \text{number_of_joints}$

5.2.2 Adaptive Robust Loss (ARL) [20]

The Adaptive Robust loss is defined as:

$$\ell_{\text{arobust}}(x) = \frac{|\alpha - 2|}{\alpha} \left(\left(\frac{|x/c|^{\gamma - g}}{|\alpha - 2|} + 1 \right)^{\alpha/2} - 1 \right), \quad [20] \tag{9}$$

where x is the difference between predicted and ground truth heatmaps, α controls the sensitivity to outliers, c sets the point where the loss transitions from a quadratic to a linear behavior, and γ determines how smooth the loss function is, especially when the predicted heatmap is close to the ground truth heatmap g. This ensures a smooth transition between small and large errors. The loss values are adaptively adjusted based on the ground truth heatmap, ensuring that small errors receive appropriate emphasis according to the context. Errors in background pixels, where the heatmap has lower values, are weighted less than those in foreground pixels, where the heatmap is more prominent. Moreover, in contrast to the MSE, the ARL is insensitive towards outliers [20].
This loss was originally developed for facial landmark detection. Given the similarities of facial landmark detection and pose estimation, it might as well be a suitable for the task at hand. For this experiment, the parameters were chosen in the following way: $\alpha = 1.0$, $c = 1.0$, $\gamma = 2.1$

5.2.3 Loss Function Results

Table 2: PCKh@0.5 Scores: Loss Functions

Method	Head	Shoulder	Elbow	Wrist	Hip	Knee	Ankle	UBody	Total	AUC
MSE(Train)	97.5	96.6	90.6	85.8	92.8	85.1	79.5	91.0	90.3	58.2
MSE (Val)	92.7	86.6	76.5	69.4	75.1	67.1	62.5	77.6	76.5	44.6
ARL (Train)	99.3	99.5	99.1	98.3	99.3	95.7	92.1	99.0	97.9	76.4
ARL (Val)	93.8	87.9	77.1	70.8	76.3	69.7	64.9	78.6	78.0	46.3

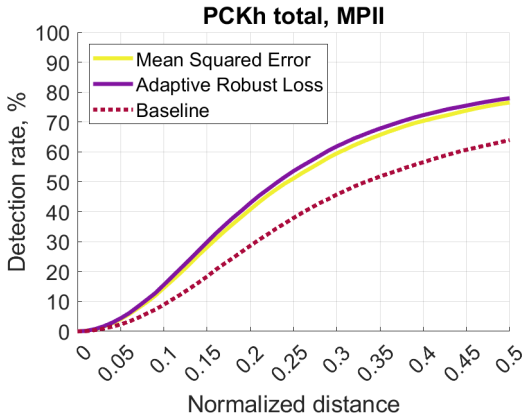


Figure 6: Loss Function Results (Val): PCKh scores at varying thresholds

On the validation set, ARL demonstrates a modest improvement over MSE, with ARL scoring higher across all joints. However, the margins are relatively small. While optimization of the ARL-parameters might further enhance the results, MSE continues to be a strong contender due to its simplicity and effectiveness. On the training set, ARL significantly outperforms MSE, but the relatively small gains on the validation set suggest overfitting.

5.3 Which postprocessing techniques are appropriate for HPE

Effective postprocessing methods could enhance the accuracy and reliability of pose estimation models. This section covers two critical areas for improvement. The implementation of these methods is described in more detail in Section 3.5.

- **Enhancing Heatmap Quality**:
  - **Gaussian Smoothing**: Applying Gaussian smoothing to predicted heatmaps could reduce noise and improve peak detection by merging closely spaced peaks of similar intensity. Slightly blurring the heatmaps could help ensuring a more accurate prediction by preventing the selection of potentially misleading peaks. In this case, a slight Gaussian blur is applied to the heatmaps with $\sigma = 1$
- **Refining Coordinate Extraction**:
  - **Pixel Refinement**: Should improve the joint location accuracy by adjusting the coordinate from the highest heatmap value to a position slightly offset towards the second-highest value [21].

5.3.1 Postprocessing Results

Table 3: PCKh@0.5 Scores: Postprocessing Techniques

Method	Head	Shoulder	Elbow	Wrist	Hip	Knee	Ankle	UBody	Total	AUC
No Postprocessing (Train)	99.3	99.5	99.1	98.3	99.3	95.7	92.1	99.0	97.9	76.4
No Postprocessing (Val)	93.8	87.9	77.1	70.8	76.3	69.7	64.9	78.6	78.0	46.3
Gaussian Smoothing (Train)	98.4	99.0	97.3	97.1	98.7	92.9	89.2	97.8	96.5	74.3
Gaussian Smoothing (Val)	91.3	83.2	66.2	59.4	70.6	57.4	54.6	69.6	69.9	42.0
Pixel Refinement (Train)	96.5	96.4	93.6	90.5	96.3	90.0	83.0	93.5	92.8	60.8
Pixel Refinement (Val)	94.0	88.0	77.4	71.2	76.8	69.9	65.4	78.8	78.3	46.8

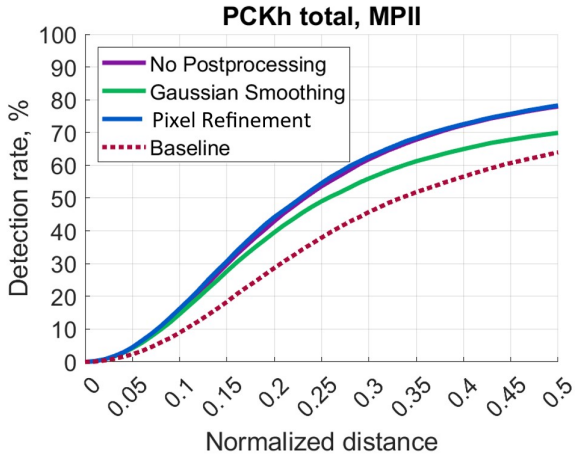


Figure 7: Postprocessing Results (Val): PCKh scores at varying thresholds

The results show that Gaussian smoothing of heatmaps negatively impacted the results on the validation and training set. Reasons for this might be that crucial peak details, which are already extracted quite well by the model, are blurred, resulting in an inaccurate localization of the joints. The pixel refinement had a slight positive impact on the validation results. It acts as a subtle form of interpolation, adjusting coordinates to correct small inaccuracies without drastically altering results. Interestingly, it also seems to help generalize the predictions, as the gain in validation score came at the cost of training accuracy.

In summary, the most effective model is the one trained using bounding boxes extracted by YOLO, optimized with Adaptive Robust Loss (ARL), and postprocessed with pixel refinement. Figure 8 illustrates examples of predicted heatmaps and coordinates from this method. Another conclusion one can draw from these results is that the model is struggling to generalize, as indicated by the gap between validation and training performance. This may be due to the intentionally chosen simplistic model architecture. Solutions for alleviating this issue could include using a more complex model or extensive data augmentation, which was not done in this project because of limited resources.



Figure 8: Example Predictions on the MPII validation/test images

6 Conclusion and Outlook

6.1 Conclusion

The results obtained from the experiments indicate that the choice of preprocessing methods significantly impacts the accuracy. Choosing YOLO for finding bounding boxes remarkably increased the the prediction accuracy compared to roughly estimated bounding boxes.

The investigation into loss functions revealed that the Mean Squared Error (MSE) is widely favored for many pose estimation tasks. Despite its simplicity, MSE proves to be effective, particularly because it penalizes larger errors more significantly, which encourages precise keypoint localization. This makes MSE well-suited for heatmap-based approaches. Training with the Adaptive Robust Loss function yielded slightly better results on training and validation set. ARL is specifically optimized for landmark detection and offers greater flexibility due to its adjustable parameters, making it a strong performer in this task as well.

The analysis showed that the postprocessing steps did not significantly enhance the overall results. Although using pixel refinement provided slightly better outcomes on the validation set than not using any postprocessing, applying Gaussian smoothing worsened the results.

In summary, aside from the model architecture, preprocessing techniques and correctly identifying bounding boxes were found to be the most important factor in achieving accurate results.

6.2 Outlook

While heatmap regression is effective for pose estimation, more advanced methods offer better results but require complex architectures and significant resources. Because of these constraints, heatmap regression was chosen, which, despite its simplicity, still delivers satisfactory results with potential for further improvement:

Expanding the model to 3D pose estimation could significantly enhance its applications, as this is a growing area of research with promising developments. For instance, models like MotionBERT [22], which leverage a transformer-based architecture, are highly effective at interpreting 3D motions from 2D data.

Optimizing the model for real-time pose estimation could benefit fields like sports analytics, live surveillance, and motion capture. The VIBE model [23] exemplifies this progress, achieving 3D pose estimation at up to 30 fps, making it ideal for real-time use.

When applying the model to video data, where it predicts poses on a frame-by-frame basis, incorporating temporal smoothing algorithms would be beneficial. This would help produce more stable and consistent results, as the model could better account for motion continuity over time.

The model currently struggles with occlusions and complex backgrounds, which reduce its accuracy. Improving its ability to handle these challenges could significantly boost performance and reliability in diverse environments.

References

[1] C. Zheng, W. Wu, T. Yang, S. Zhu, C. Chen, R. Liu, J. Shen, N. Kehtarnavaz, and M. Shah, "Deep learning-based human pose estimation: A survey," *CoRR*, vol. abs/2012.13392, 2020. [Online]. Available: https://arxiv.org/abs/2012.13392

[2] S. Dubey and M. Dixit, "A comprehensive survey on human pose estimation approaches," *Multimedia Systems*, vol. 29, 2022.

[3] R. Pytel, "Human pose estimation 2023 guide," 2023, accessed: 2024-08-16. [Online]. Available: https://softwaremill.com/human-pose-estimation-2023-guide/

[4] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, "2d human pose estimation: New benchmark and state of the art analysis," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[6] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *ArXiv*, vol. abs/1511.08458, 2015. [Online]. Available: https://api.semanticscholar.org/CorpusID:9398408

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[8] Q. Liu and Y. Wu, *Supervised Learning*. Boston, MA: Springer US, 2012, pp. 3243–3245. [Online]. Available: https://doi.org/10.1007/978-1-4419-1428-6_451

[9] B. Xiao, H. Wu, and Y. Wei, "Simple baselines for human pose estimation and tracking," in *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.

[10] P. Gavrikov, "visualkeras," https://github.com/paulgavrikov/visualkeras, 2020, accessed: 2024-08-27.

[11] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, "Efficient object localization using convolutional networks," *CoRR*, vol. abs/1411.4280, 2014. [Online]. Available: http://arxiv.org/abs/1411.4280

[12] L. Neumann and A. Vedaldi, "Tiny people pose," in *Asian Conference on Computer Vision*, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:67790080

[13] V. Belagiannis and A. Zisserman, "Recurrent human pose estimation," in *2017 12th IEEE International Conference on Automatic Face Gesture Recognition (FG 2017)*, 2017, pp. 468–475.

[14] U. Rafi, B. Leibe, J. Gall, and I. Kostrikov, "An efficient convolutional network for human pose estimation," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2016, pp. 109.1–109.11. [Online]. Available: https://dx.doi.org/10.5244/C.30.109

[15] T. D. Nguyen and M. Kresovic, "A survey of top-down approaches for human pose estimation," *ArXiv*, vol. abs/2202.02656, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:246634325

[16] R. Cheng, "A survey: Comparison between convolutional neural network and yolo in image identification," *Journal of Physics: Conference Series*, vol. 1453, p. 012139, 2020.

[17] M. Nixon and A. Aguado, *Feature Extraction and Image Processing*. Academic, 2008. [Online]. Available: https://books.google.at/books?id=jXmJqzQgdY8C

[18] A. Nibali, "eval-mpii-pose," https://github.com/anibali/eval-mpii-pose, 2017, accessed: 2024-08-28.

[19] T. L. Munea, Y. Z. Jembre, H. T. Weldegebriel, L. Chen, C. Huang, and C. Yang, "The progress of human pose estimation: A survey and taxonomy of models applied in 2d human pose estimation," *IEEE Access*, vol. 8, pp. 133 330–133 348, 2020.

[20] Y. Tian, D. Su, and S. Li, "Adaptive robust loss for landmark detection," *Information Fusion*, vol. 101, p. 102013, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253523003299

[21] B. Xiao, H. Wu, and Y. Wei, "Simple baselines for human pose estimation and tracking," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[22] W. Zhu, X. Ma, Z. Liu, L. Liu, W. Wu, and Y. Wang, "Motionbert: A unified perspective on learning human motion representations," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[23] M. Kocabas, N. Athanasiou, and M. J. Black, "Vibe: Video inference for human body pose and shape estimation," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.